

Optimization of Multi-AGV Scheduling for Airport Baggage Handling

Abdallah Ahmed Hassan¹, Abdelrahman Ewida¹, Haidy Ehab Elkenawy¹,
Abdulrahman Bassem Bahy¹, Karim Mohamed Fathy¹, Dalia M. Mahfouz^{2,3} and Omar M. Shehata^{2,3}

Abstract—Efficient coordination of Automated Guided Vehicles (AGVs) is critical for mitigating the multi-billion dollar operational losses associated with airport baggage mishandling. This paper presents a multi-objective optimization framework for AGV scheduling, formulated as a Vehicle Routing Problem with Time Windows (VRPTW) on a bidirectional linear topology. We evaluate five distinct optimization paradigms: three established meta-heuristics (Simulated Annealing (SA), Genetic Algorithm (GA), and Ant Colony Optimization (ACO)), a proposed Hybrid Artificial Bee Colony (ABC) algorithm, and a Reinforcement Learning (RL) (Q-Learning) approach. The Hybrid ABC approach utilizes discrete neighborhood operators for routing and continuous vector updates for temporal scheduling, while the Q-Learning agent offers sub-second responsiveness for dynamic environments. Experiments across three operational scenarios (standard, small, and a high-pressure First Come First Serve (FCFS) instance) demonstrate that the Hybrid ABC algorithm achieves the highest optimality, improving upon the traditional FCFS method by 61% and reducing fleet requirements by 40% in the Standard instance. Conversely, the Q-Learning agent provides the highest execution efficiency, establishing a clear trade-off between planning quality and real-time responsiveness.

Index Terms—AGV Scheduling, ABC, Meta-heuristics, RL, Airport Logistics, Optimization.

I. INTRODUCTION

As airports increasingly operate as complex, high-density ecosystems, autonomous ground handling has emerged as a critical domain for enhancing efficiency and scalability. Global passenger traffic has surpassed **5.2 billion annually** [1], placing immense pressure on legacy scheduling systems where delays cost the industry over **\$33 billion per year** [2]. Specifically, baggage mishandling incurs annual losses exceeding **\$5 billion** due to the limitations of manual dispatching. Consequently, the transition to intelligent AGV coordination is not merely a technological upgrade but a financial necessity to mitigate these multi-billion dollar operational losses.

Recent academic discourse on airport ground handling has transitioned from simple heuristic dispatching to complex, multi-objective evolutionary frameworks. A primary focus of contemporary research is the interdependence between aircraft positioning and vehicle routing, which manual scheduling approaches fail to address adequately.

Yao et al. [3] argued that these elements must be solved in a coupled manner rather than in isolation. To address constraints related to flight-stand compatibility and strict task coordination, they proposed a two-stage optimization model using a hybrid Non-dominated Sorting Genetic Algorithm II (NSGA-II).

While their results demonstrated that accounting for this coupling minimizes taxi fuel consumption compared to decoupled strategies, their model assumes deterministic service times. This rigidity often fails to account for the stochastic nature of flight delays, leading to schedule infeasibility when strict time window constraints are violated by external factors.

As airport operations increasingly adopt sustainable technologies, standard routing models have been expanded to accommodate Electric Ground-Handling Vehicles (E-GHVs). Fu et al. [4] focused on the distinct constraints of battery range limitations and non-linear charging intervals, which complicate standard vehicle availability. They formulated a bi-objective integer programming model designed to balance operational efficiency with workload fairness, solved via a hybrid Large Neighborhood Search (LNS).

Although their approach ensures feasible operations under energy limitations, the high computational complexity of the non-linear charging model makes it less suitable for dynamic recovery. In scenarios requiring immediate re-routing due to capacity overflows, their method lacks the sub-second responsiveness required for real-time operations.

Specific attention has also been directed toward the handling of irregular demands in cargo and baggage logistics. Li et al. [5] targeted the scheduling of AGVs for irregular air cargo, utilizing a Mixed-Integer Linear Programming (MILP) formulation. To handle constraints regarding single-tasking exclusivity and vehicle load capacity ($w_i \cdot x_{ik} \leq Cap_k$), they implemented a hybrid Simulated Annealing Genetic Algorithm (SAGA). Similarly, in the specific domain of baggage transport, Guo et al. [6] addressed constraints concerning depot return requirements and strict service windows.

They proposed a diversity-enhanced Genetic Algorithm (GA-EFD) to tackle the issue of premature convergence. However, despite the robustness of these hybrid meta-heuristics, the underlying Genetic Algorithm framework remains inherently susceptible to local optima traps [5].

This theoretical limitation necessitates more aggressive local search mechanisms, such as the scout bee phase in ABC to ensure the algorithm can continuously refine the solution space and achieve the global optimum for fleet size minimization.

¹Faculty of Media Engineering and Technology (MET), German University in Cairo, Egypt, {abdallah.ahmed, abdelrahman.oweida, haidy.abdelsalam, abdulrahman.awad, karim.abdelhafeez}@student.guc.edu.eg

²Mechatronics Dept., Faculty of Engineering and Materials Science (EMS), German University in Cairo, Egypt, {dalia.mahfoz, omar.mohamad}@guc.edu.eg

³Multi-Robot Systems (MRS) Research Group, Cairo, Egypt

TABLE I: Key differences between related works and the proposed approach

Feature	Yao [3]	Fu [4]	Li [5]	Guo [6]	Luo [7]	This work
Time windows	✓	×	✓	✓	✓	✓
Capacity	×	×	✓	✓	×	✓
Fleet size obj.	×	×	×	✓	×	✓
Digital Twin	×	×	×	×	✓	×
Baseline (SA/GA/ACO)	×	×	×	×	×	✓
ABC (scout reset)	×	×	×	×	×	✓
Q-Learning policy	×	×	×	×	×	✓

Finally, the field is advancing toward handling stochastic uncertainty through dynamic simulation integration. Luo et al. [7] moved beyond static optimization by addressing complex constraints such as inter-vehicle precedence via a Digital Twin (DT) framework. This approach utilizes real-time feedback to validate algorithmic decisions against simulated anomalies. While effective for robustness, the reliance on a constant, high-fidelity DT data stream introduces significant infrastructure overhead and a potential single point of failure. This creates a clear research gap: the need for a lightweight autonomous decision-making agent that can construct feasible schedules without the computational overhead of repeatedly re-running complex meta-heuristics.

To further emphasize the contribution of this work, a comprehensive performance benchmark of the standard meta-heuristic approaches (SA, GA, ACO) and the proposed ABC and RL strategies is presented in Table I.

The remainder of this paper is organized as follows: Section II defines the mathematical formulation of the AGV scheduling problem, detailing the objective function and operational constraints. Section III describes the implementation of the five comparative algorithms. Section IV presents the experimental results. Finally, Section V concludes the work and outlines potential future research directions.

II. PROBLEM FORMULATION

In this study, we model the airport baggage handling process as a multi-objective VRPTW on a linear bidirectional path. The system comprises a set of N transportation tasks and a heterogeneous fleet of M AGVs. Each task i is characterized by its pickup and delivery locations, a payload weight w_i , a processing duration p_i , and a strict time window $[ES_i, LS_i]$.

The optimization objective is to minimize a composite fitness function $F(x)$, which aggregates competing operational metrics:

$$\min F(x) = w_d D_{\text{total}} + w_t Z_{\text{sum-start}} + w_f Z_{\text{fleet}} + w_v Z_{\text{workload-var}} \quad (1)$$

These weights reflect a predefined operational prioritization of scheduling speed and fleet cost over secondary metrics like workload variance; a detailed parametric study of their effects is reserved for future work. Where D_{total} is the total distance traveled, and $w_{d,t,f,v}$ are user-defined weights. The detailed formulations for the remaining operational metrics are defined below.

A. Minimize Sum of Task Start Times

This objective minimizes the total start times of all tasks across the AGV fleet, thereby reducing the overall operational duration of the system.

Since each task has a constant service time, minimizing the sum of start times equivalently minimizes the corresponding completion times, resulting in improved system efficiency.

$$\min Z_{\text{sum-start}} = \sum_{k \in K} \sum_{i \in I} x_{ik} s_{ik} \quad (2)$$

B. Minimize Number of AGVs Used

To formulate the objective of minimizing the fleet size for a given set of tasks, we use an auxiliary binary decision variable, u_k . This variable takes a value of 1 if AGV k is assigned at least one task, and 0 otherwise. The objective function is to minimize the sum of these variables.

$$\min Z_{\text{fleet}} = \sum_{k \in K} u_k \quad (3)$$

C. Balance AGV Workload

This objective minimizes the squared deviation of each AGV's workload from the average, encouraging a balanced distribution of tasks across all AGVs.

$$\min Z_{\text{workload-var}} = \sum_{k \in K} \left(L_k - \frac{T}{|K|} \right)^2 \quad (4)$$

Explanation:

- $L_k = \sum_{i \in I} x_{ik}$ represents the total number of tasks assigned to AGV k .
- $\frac{T}{|K|}$ is the average number of tasks per AGV, where $T = \sum_{i \in I} 1$ denotes the total number of tasks.

D. Maximize Number of Tasks per AGV

The third objective aims to maximize the utilization of AGVs by increasing the number of assigned tasks. Using the binary assignment variable x_{ik} , this objective is formulated as:

$$\max Z_{\text{tasks}} = \sum_{k \in K} \sum_{i \in I} x_{ik}. \quad (5)$$

This encourages efficient AGV usage by minimizing idle vehicles and distributing workload effectively.

E. Constraints

The solution space is governed by strict physical and operational limits. A valid schedule must satisfy the following constraints:

AGV Capacity Constraint: This constraint enforces the physical limitations of the AGV fleet. It ensures that the weight of the baggage for any assigned task (w_i) does not exceed the maximum carrying capacity of the AGV to which it is assigned (Cap_k).

$$w_i \cdot x_{ik} \leq Cap_k, \quad \forall i \in I, \forall k \in K$$

Fleet Size Constraint: This constraint reflects the physical limitation of the system by ensuring that the number of AGVs

used in any valid solution does not exceed the total number of AGVs available in the fleet ($|K|$).

$$\sum_{k \in K} u_k \leq |K|$$

Time Window Constraint: This constraint is essential for maintaining a high level of service. It guarantees that each task is handled within its specific operational window, starting no earlier than its ready time (ES_i) and completing no later than its due date (LS_i).

$$ES_i \leq s_{ik} \leq LS_i - p_i, \quad \forall i \in I, \forall k \in K$$

To validate the proposed formulation, we defined three distinct operational scenarios ranging from standard daily operations to critical resource scarcity. Table II summarizes the specific parameters (task count, fleet size, and aircraft volume) for each case study.

The spatial layout across instances is modeled as a 7-node graph comprising a depot, five aircraft gates, and a central warehouse (an example of the Small Instance layout, featuring three gates, is shown in Fig. 1). Travel costs are computed via Manhattan distance. The task set includes mixed LOAD/UNLOAD operations with weights ranging from 800 kg (Light) to 3000 kg (Heavy), reflecting actual aircraft capacities. Temporal constraints are enforced through staggered time windows ($ES \in [0, 180]$, $LS \in [200, 480]$) and variable processing times (5–8 units), ensuring a rigorous test of each algorithm’s ability to maintain schedule feasibility under diverse operational loads.

The spatial layout across instances is modeled as a bi-directional graph comprising a depot, up to five aircraft gates, and a central warehouse (an example of the Small Instance layout, featuring three gates, is shown in Fig. 1). Travel costs are computed via Manhattan distance.

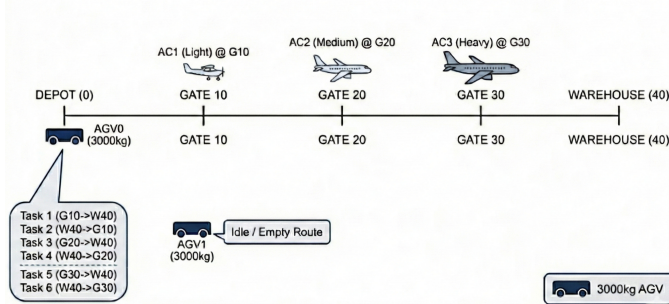


Fig. 1: Schematic of Case Study 2 (Small Instance), illustrating the linear airport topology with three Aircraft Gates.

TABLE II: Operational Parameters of the Three Case Studies

Case Study	Tasks	AGVs	Aircrafts
Standard	12	11	5
Small	6	2	3
FCFS	16	2	8

III. METHODOLOGY

This section details the diverse set of algorithms implemented to solve the formulated problem. We begin by summarizing the three baseline meta-heuristics developed in prior phases of this study, followed by a detailed description of the novel Hybrid ABC algorithm and the Q-Learning agent.

A. Baseline Algorithms

To establish a rigorous performance benchmark, three optimization paradigms were implemented using the parameters summarized below:

1) *Simulated Annealing (SA)* [8]: Is a trajectory-based algorithm that escapes local optima by probabilistically accepting inferior solutions based on the Metropolis criterion ($P = e^{-\Delta f/T}$). To ensure consistent neighborhood exploration across methods, our implementation utilizes the neighbor generation operators defined in **Algorithm 1**.

A geometric cooling schedule governed by $T_{k+1} = \alpha T_k$ was adopted to balance exploration and exploitation. The baseline configuration utilizes an initial temperature $T_0 = 1000$, a slow cooling rate $\alpha = 0.999$, and a Markov chain length of $N = 1$.

2) *Genetic Algorithm (GA)* [9]: Is a population-based evolutionary strategy. We designed a two-part chromosome representation encoding both *Task Assignment* and *Task Sequence*. The evolution process utilizes **Tournament Selection** to choose parents, followed by **Ordered Crossover (OX1)** with a probability of $p_c = 0.8$ to preserve sequence validity. To maintain diversity, mutation is applied using the operators from **Algorithm 1** with a rate of $p_m = 0.1$. The baseline configuration operates with a population size of $N_{pop} = 100$ over $N_{gen} = 50$ generations, retaining the top 10% of solutions (Elitism).

3) *Ant Colony Optimization (ACO)* [10]: Is a constructive swarm intelligence method where artificial ants build solutions probabilistically. Our implementation models the problem as a graph traversal where ants select the next task based on a combination of heuristic desirability ($\eta_{ij} = 1/d_{ij}$) and pheromone trails (τ_{ij}). The baseline configuration balances these factors equally with importance weights $\alpha = 1.0$ and $\beta = 1.0$. The search is conducted by a colony of $N_{ants} = 50$ agents over $N_{iter} = 100$ iterations. Pheromone trails are globally updated using an evaporation rate of $\rho = 0.5$ and a deposition constant of $Q = 100.0$.

B. Proposed Algorithm: Hybrid Artificial Bee Colony (ABC)

To address the discrete nature of the AGV routing problem, we adapted the standard ABC algorithm. The solution vector comprises discrete route sequences and continuous start times. The algorithm proceeds in three phases:

- 1) **Initialization:** A population of N feasible solutions is generated using a capacity-aware heuristic.
- 2) **Search (Employed & Onlooker Bees):** Both bee types explore the search space by generating neighbor solutions. For the discrete routing component, they utilize the neighbor generation operators defined in **Algorithm**

Algorithm 1 Neighbor Generation Strategy

```

1:  $S' \leftarrow S_{\text{curr}}$ 
2:  $op \leftarrow \text{Random}(\{\text{Move}, \text{SwapInter}, \text{SwapIntra}\})$ 
3: if  $op = \text{Move}$  then
4:   Select random AGV  $k_{\text{src}}$  with tasks and  $k_{\text{dest}}$  ( $k_{\text{src}} \neq k_{\text{dest}}$ )
5:   Move random task  $t$  from  $S'[k_{\text{src}}]$  to  $S'[k_{\text{dest}}]$ 
6: else if  $op = \text{SwapInter}$  then
7:   Select AGVs  $k_1, k_2$  and tasks  $t_1 \in S'[k_1], t_2 \in S'[k_2]$ 
8:   Swap assignments of  $t_1$  and  $t_2$ 
9: else if  $op = \text{SwapIntra}$  then
10:  Select AGV  $k$  with  $|S'[k]| \geq 2$ 
11:  Swap positions of two random tasks  $t_a, t_b \in S'[k]$ 
12: end if
13: return  $S'$ 

```

1. For the continuous start times (S), the standard ABC perturbation is applied:

$$v_{ij} = x_{ij} + \phi_{ij}(x_{ij} - x_{kj}) \quad (6)$$

where $\phi \in [-1, 1]$. Onlooker bees select solutions to improve based on Roulette Wheel Selection.

- 3) **Reset (Scout Bees):** To prevent stagnation in local optima, any solution that fails to improve after a predefined limit of trials is abandoned and replaced by a new randomly generated feasible solution.

C. Neighborhood Structures

To ensure a fair comparison, both the ABC and SA algorithms utilize the same set of neighborhood operators to generate candidate solutions. This ensures that performance differences are due to the search strategy (meta-heuristic), not the local move definition. Algorithm 1 outlines the three operators used.

D. Reinforcement Learning (Q-Learning)

To contrast the static scheduling approach of meta-heuristics with dynamic decision-making, we implemented a Q-Learning agent. The agent constructs the schedule sequentially by learning a policy $\pi(s)$ that maps states to optimal actions.

- **State Representation (S):** The state is defined by the current physical location of the AGV (node index).
- **Action Space (A):** The set of unvisited Task IDs that satisfy capacity and time window constraints relative to the current state.
- **Reward Function (R):** To align with the global objective of minimizing distance, the immediate reward is defined as:

$$R(s, a) = \frac{100}{d(s, s') + 1} \quad (7)$$

The agent learns the optimal action-value function $Q(s, a)$ through an ϵ -greedy exploration strategy over 1,000 training episodes.

IV. RESULTS

This section presents the empirical assessment of five algorithms (SA, GA, ACO, ABC, and Q-Learning (RL)) on AGV task assignment and scheduling. All experiments were

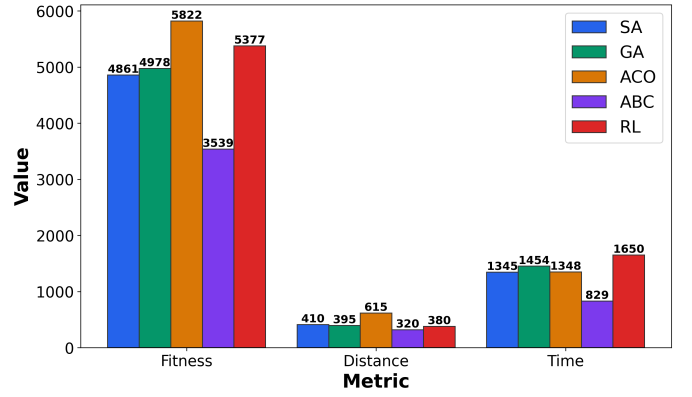


Fig. 2: Comparative analysis of different metrics across the five implemented algorithms. The Hybrid ABC algorithm demonstrates superior performance across all metrics.

implemented in Python 3.12 and executed on an Apple M1 Pro workstation with 16 GB unified memory to ensure consistent benchmarking. The algorithms were evaluated on three case studies: the Standard instance, the Small instance, and the FCFS instance.

Because the meta-heuristics and RL are stochastic, we report results from single representative runs for the Standard instance for figures, and we use the comprehensive performance table (Table III) for cross-instance and cross-algorithm comparison. The subsection below first summarizes overall performance via bar charts and convergence, then stability, workload balance, and finally algorithm-specific validation and discussion. Figure 1 shows the layout of the Small instance (Case Study 2), with a linear airport topology and 6 tasks. The Standard (feasible) instance used for most comparative figures comprises 12 tasks and 11 AGVs; the FCFS instance models a strike scenario with 16 tasks and 2 AGVs.

A. Overall Performance: Bar Comparison and Convergence

Figure 2 summarizes the final fitness, total travel distance, and total operational time for all five algorithms on the Standard instance. ABC achieves the best performance on all three metrics: the lowest fitness (3539), the shortest total distance (320), and the shortest operational time (829).

ACO attains the highest fitness (5822) and the longest distance (615), while Q-Learning (RL) yields the highest operational time (1650) despite a moderate fitness (5377) and distance (380). SA and GA are comparable, placing them between ABC and the weaker performers.

Figure 3 compares the convergence behavior of all five algorithms. The ABC algorithm (blue line) exhibits a distinct stepped descent: after a rapid initial improvement from about 5050, it undergoes a sharp breakthrough from the starting point up to 40% completion (fitness dropping from about 5050 to 3850), then continues to improve to a final best fitness of approximately 3520. This pattern validates the effectiveness of the *Scout Bee* mechanism in escaping local optima. In contrast, the GA (green line) plateaus early, by about 35% completion,

TABLE III: Comprehensive Performance Comparison Across All Case Studies

Case Study	Algorithm	Distance	Op. Time	Fleet	Variance	Total Fitness	Exec. Time
Standard	Initial	890	246	11	7452.98	6157.55	-
	SA	410	1296	4	28.91	4782.55	~38s
	GA	400	1444	4	24.91	5008.55	~4s
	ACO	605	1359.50	10	2.91	5784.55	~10s
	RL	370	1622.50	3	36.91	5270.55	~1s
	ABC	335	839.69	3	42.91	3559.93	~143s
Small	Initial	240	243	2	14762.25	2443.00	-
	SA	170	393.50	2	0	1635	~11s
	GA	170	393.50	2	0	1635	~2s
	ACO	190	381	2	0	1710	~3s
	RL	140	564	1	18	1911	~1s
	ABC	130	310.00	1	18	1353.00	~42s
FCFS	Initial	900	919	2	10100.25	20548.00	-
	SA	490	2806	2	8	8080	~70s
	GA	520	2919.50	2	2	8427	~4s
	ACO	470	2889	2	0	8106	~12s
	RL	610	3630	1	128	10923	~3s
	ABC	470	2856	2	8	8080	~250s

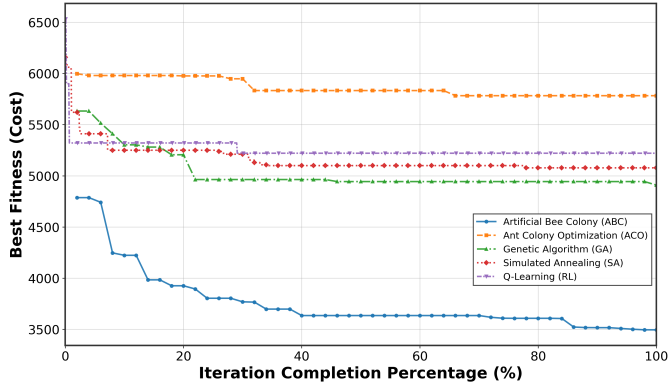


Fig. 3: Comparative Convergence: ABC outperforms SA, GA, ACO, and Q-Learning algorithms.

at a fitness around 5150–5170. SA (red line) improves in stages and reaches its final plateau (about 4950–5000) after roughly 40% completion, while Q-Learning RL (purple line) stabilizes relatively early at around 5580–5600.

B. Stability and Workload Distribution

To assess robustness, we ran each algorithm 10 times on the Standard instance. Figure 4 shows the fitness distribution, where ABC and ACO display tighter inter-quartile ranges, indicating superior stability compared to GA and SA.

Workload balance is quantified via the variance metric in Table III. ACO achieves the lowest variance (2.91), indicating a near-perfectly uniform distribution of tasks. However, this comes at the cost of using 10 AGVs. The proposed ABC algorithm maintains a competitive balance (Variance: 42.91) while using only 3 AGVs, proving it can optimize fleet size without creating bottlenecks on specific vehicles.

Figure 5 illustrates the AGV workload distribution. While ACO (Fig. 5a) achieves optimal balance, it inefficiently deploys 10 AGVs for 12 tasks, causing high idle times. Con-

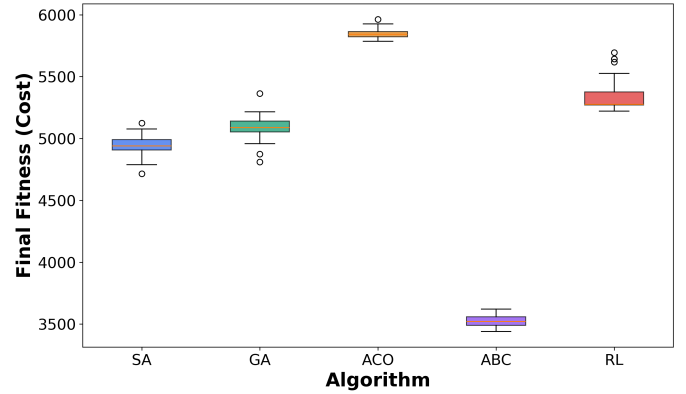


Fig. 4: Distribution of final fitness over 25 runs per algorithm on the Standard instance.

versely, the Q-Learning agent (Fig. 5d) maximizes resource consolidation by requiring only 3 AGVs, and Hybrid ABC (Fig. 5e) utilizes 4. SA (Fig. 5c) exhibits severe imbalance, assigning over 58% of tasks to a single vehicle. Ultimately, RL and ABC achieve compact fleet sizes comparable to GA, but reach higher global optimality by significantly reducing total distance and operational time.

C. Comparative Analysis

Table III summarizes final performance across all three case studies. Metrics include total distance, operational time, fleet size, workload variance, total fitness, and execution time. While GA, SA, and RL stabilize by approximately 40% of the cycle, ABC and ACO exhibit late-stage breakthroughs at 85% and 65% respectively. This highlights the effectiveness of the ABC scout phase in escaping local optima compared to trajectory-based baselines.

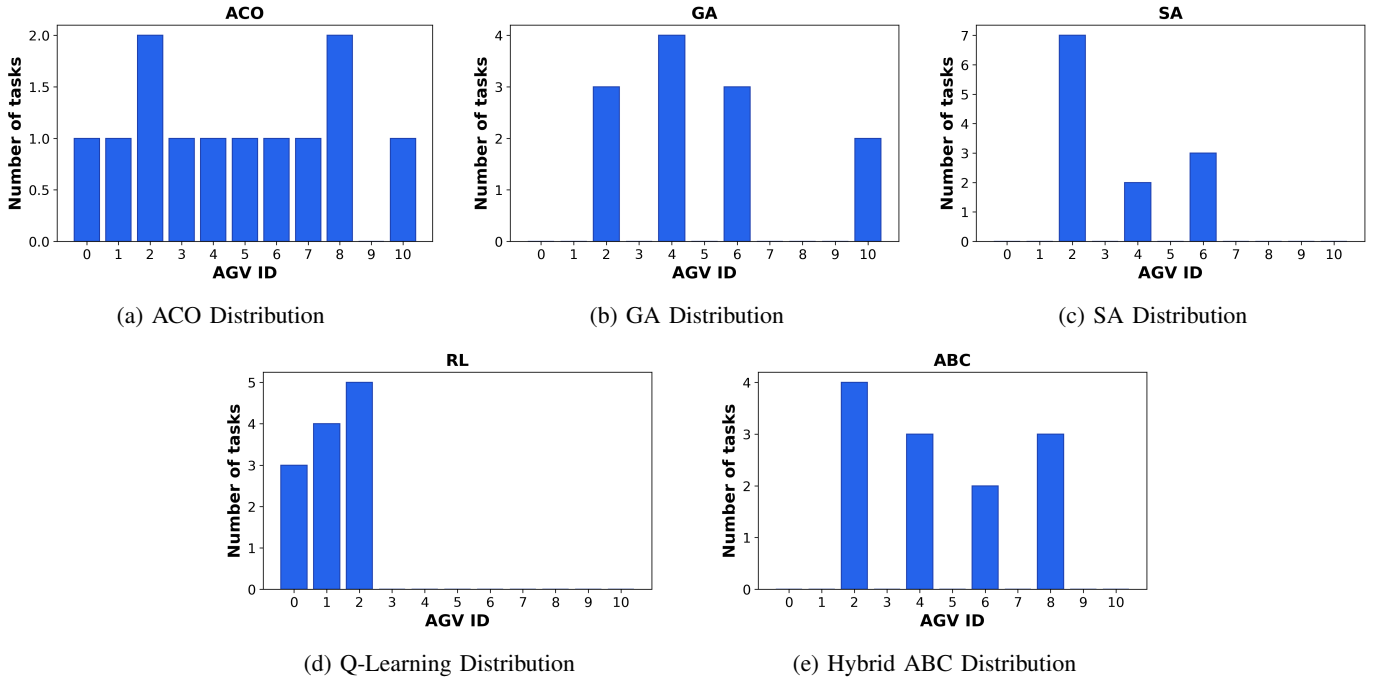


Fig. 5: Workload distribution analysis across the AGV fleet. The top row compares baseline meta-heuristics, while the bottom row highlights the proposed learning and hybrid approaches.

V. CONCLUSION

This study established a comprehensive benchmark for airport baggage AGV scheduling, comparing five optimization architectures across varying operational pressures. The results validate that the proposed Hybrid ABC algorithm is the superior strategy for offline planning, consistently achieving the lowest global fitness and most efficient fleet utilization. Specifically, the ABC algorithm improved operational efficiency by approximately **61%** over the traditional FCFS method and outperformed established meta-heuristics like SA and GA in fleet size minimization. However, the analysis revealed a critical performance-latency trade-off: while ABC provides near-optimal schedules, its computational overhead (~ 143 s) limits its utility in high-frequency dynamic environments. In contrast, the Q-Learning agent demonstrated instantaneous execution (~ 1 s) despite lower solution optimality. We conclude that a hierarchical operational model through deploying ABC for daily static planning and RL-based agents for real-time disruption recovery. It represents the most viable path toward fully autonomous ground handling.

Future work will expand this framework by incorporating dynamic obstacle avoidance for passenger-shared tarmac environments and non-linear battery charging cycles. Furthermore, we intend to implement Deep Reinforcement Learning (DRL) architectures, such as DQN or PPO, to bridge the current optimality gap and enable precise timing adjustments within continuous state spaces, alongside a detailed parametric study on the fitness function weighting factors.

REFERENCES

- [1] SITA, "Baggage it insights 2024," Société Internationale de Télécommunications Aéronautiques, Tech. Rep., June 2024, reports global baggage mishandling costs at \$5 billion and passenger traffic at 5.2 billion. [Online]. Available: <https://www.sita.aero/resources/surveys-reports/baggage-it-insights-2024>
- [2] Airlines for America, "U.s. passenger carrier delay costs," <https://www.airlines.org/dataset/u-s-passenger-carrier-delay-costs/>, 2024, estimated annual cost of flight delays exceeds \$33 billion including direct and indirect costs.
- [3] M. Yao, M. Hu, J. Yin, J. Su, and M. Yin, "A two-stage optimization model for airport stand allocation and ground support vehicle scheduling," *Applied Sciences*, vol. 14, no. 23, p. 11407, Dec. 2024. [Online]. Available: <http://dx.doi.org/10.3390/app142311407>
- [4] W. Fu, J. Li, Z. Liao, and Y. Fu, "A bi-objective optimization approach for scheduling electric ground-handling vehicles in an airport," *Complex & Intelligent Systems*, vol. 11, no. 4, Mar. 2025. [Online]. Available: <http://dx.doi.org/10.1007/s40747-025-01815-x>
- [5] J. Li, M. Zou, Y. Lv, and D. Sun, "Agv scheduling for optimizing irregular air cargo containers handling at airport transshipment centers," *Mathematics*, vol. 12, no. 19, p. 3045, Sep. 2024. [Online]. Available: <http://dx.doi.org/10.3390/math12193045>
- [6] W. Guo, P. Xu, Z. Zhao, L. Wang, L. Zhu, and Q. Wu, "Scheduling for airport baggage transport vehicles based on diversity enhancement genetic algorithm," *Natural Computing*, vol. 19, no. 4, p. 663–672, Aug. 2018. [Online]. Available: <http://dx.doi.org/10.1007/s11047-018-9703-0>
- [7] Q. Luo, H. Liu, C. Liu, and Q. Deng, "Multi-strategy cooperative scheduling for airport specialized vehicles based on digital twins," *Scientific Reports*, vol. 14, no. 1, Jul. 2024. [Online]. Available: <http://dx.doi.org/10.1038/s41598-024-66350-0>
- [8] S. Kirkpatrick, C. D. Gelatt, and M. P. Vecchi, "Optimization by simulated annealing," *Science*, vol. 220, no. 4598, pp. 671–680, 1983.
- [9] J. H. Holland, *Adaptation in natural and artificial systems: an introductory analysis with applications to biology, control, and artificial intelligence*. University of Michigan Press, 1975.
- [10] M. Dorigo, "Optimization, learning and natural algorithms," Ph.D. dissertation, Politecnico di Milano, 1992.